A3-PS023

Minimal Two-Node Cloud Hypervisor VM Controller

Team Name: Aachi Masala

mitumithilesh2@gmail.com

Mithilesh.B , Rithika.R

AIORI Hackathon 3

October 14, 2026

Problem Description

Edge sites need to run VMs on small KVM hosts without a heavy, multi-daemon hypervisor stack.

- Enterprise stacks (libvirt, QEMU device model, helper daemons) cost memory and operations effort that small edge hosts cannot spare
- Placing VMs without looking at real free memory overloads one host while another sits idle
- Without enforced limits, one VM can starve its neighbours of CPU or memory
- Failed API calls leave orphaned VM processes, tap devices and disks behind
- Unauthenticated host agents let anyone on the network start or kill VMs

Goal(PS-023): a central controller and two worker agents that create, start, stop and list VMs through the Cloud Hypervisor management API across two `/dev/kvm` hosts.

Background

Cloud Hypervisor is a minimal, Rust-based VMM that runs directly on Linux KVM and is controlled through a REST API.

- **KVM** turns the Linux kernel into a hypervisor; user space reaches it through `/dev/kvm`
- **Cloud Hypervisor** runs one process per VM, uses only modern virtio devices, and exposes a Unix-socket API: `vm.create`, `vm.boot`, `vm.shutdown`, `vm.info`, `vmm.shutdown`
- **rust-hypervisor-firmware** boots standard Ubuntu cloud images without a separate kernel
- **cgroup v2 via systemd** lets the kernel enforce per-VM memory, CPU and task caps
- **Constraints:** 2 KVM hosts, 1 prepared image; no live migration, distributed storage, browser VNC, image marketplace or new hypervisor

Literature & RFC Review

We reused proven standards and designs instead of inventing new protocols.

Source	What we adopted
Cloud Hypervisor API [2]	Exact lifecycle endpoints and the `VmConfig` body for `vm.create`
RFC 2104 — HMAC [6]	HMAC-SHA256 signing of every controller → worker request (method, path, timestamp, nonce, body hash)
RFC 6750 — Bearer tokens [7]	`Authorization: Bearer` API key for clients of the controller
RFC 9110 — HTTP semantics [8]	Status codes: 401 auth, 409 conflict, 422 limit violated, 502/503 worker errors
Kubernetes *LeastAllocated* scoring [9]	Spread (worst-fit) placement: pick the host with the most free memory
systemd resource control [5]	`MemoryMax`, `CPUQuota`, `TasksMax` on each VM's transient unit

Proposed Solution

A central controller places VMs; one agent per host drives Cloud Hypervisor over its Unix-socket API.

- **Controller** (FastAPI + SQLite): authenticated REST API, memory-based placement, limit checks, reconciler every 15 s
- **Worker agent** ×**2**: HMAC-verified API. Per VM: sparse disk copy + cloud-init seed, tap device on a private bridge, `cloud-hypervisor` in its own systemd unit
- **One prepared image**: Ubuntu 22.04 cloud image, booted by `hypervisor-fw`
- **Leak-free cleanup**: rollback on any failed step, agent reaper and controller reconciler every 15 s
- **Testbed**: two `qemu x/qemu` hosts with nested KVM [10]

Operation	Cloud Hypervisor call
create	`vm.create`
start	`vm.boot`
stop	`vm.power-button`, then `vm.shutdown`
list	`vm.info` per VM
delete	`vmm.shutdown` + stop unit

Optimization Proposed

Each optimization targets footprint, placement accuracy or leak-free cleanup.

Area	Before (typical approach)	After (our design)
VMM footprint	libvirtd + QEMU device model + helper daemons	One static `cloud-hypervisor` binary per VM
Placement	Round-robin / first-fit, ignores real memory	Worst-fit on min(reserved free, kernel MemAvailable)
Concurrent creates	Two requests can overbook one host	Placement lock + per-VM in-flight reservations
Resource limits	Checked once at the API	Five layers, ending in kernel cgroups and fixed CH vCPU/RAM
Disk provisioning	Full image copy per VM	Sparse / reflink copy + small NoCloud seed
Process cleanup	Stray PIDs after crashes	systemd unit per VM, `KillMode=control-group`
Failure recovery	Manual cleanup	Rollback + agent reaper + controller reconciler
Host auth	Open agent port or shared password	Per-node HMAC secret, nonce + 30 s timestamp window

Solution Architecture & Design

Controller, two worker agents and Cloud Hypervisor on two KVM hosts.

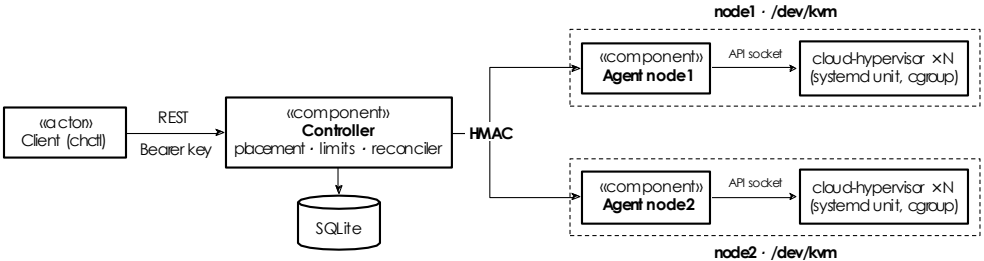


Figure: System Architecture UML Diagram

Data Flow Diagram

Create-VM request: from client to a booted Cloud Hypervisor guest.

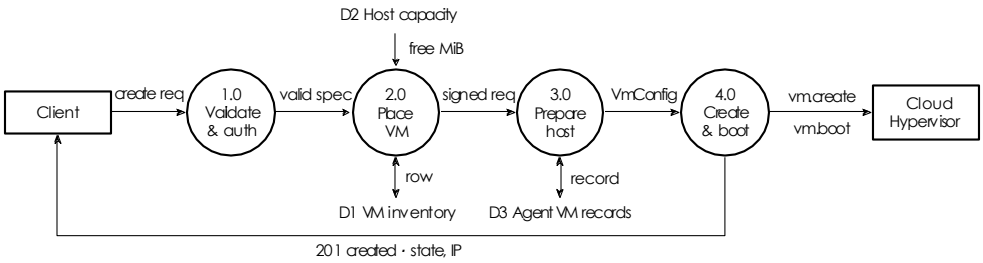


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Methodology: build and test against a simulated Cloud Hypervisor first, then deploy on nested-KVM hosts.

Date	Milestone	Deliverable	Status
12 Oct	**Analysis & design**	Architecture, APIs and placement rules fixed	Done
13 Oct	**Core build**	Controller, 2 agents, HMAC auth, limits, cleanup; 26 automated tests	Done
13 Oct	**Testbed qualified**	Nested KVM verified on an AMD EPYC OpenStack VM	Done
14 Oct	**Report**	This proposal document	Done
Next	**Live deployment**	`docker compose up`: two `qemux/qemu` nodes + controller	Planned
Next	**Validation**	`demo.sh` end to end; measure boot time and placement	Planned
Next	**Submission**	Private repo transferred to aiori-hackathon; mentor demo	Planned

Bibliography I

1 Cloud Hypervisor Project. "Cloud Hypervisor: a Virtual Machine Monitor for modern cloud workloads". GitHub repository. https://github.com/cloud-hypervisor/cloud-hypervisor.

2 Cloud Hypervisor Project. "Cloud Hypervisor API". `docs/api.md`. https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/api.md.

3 Cloud Hypervisor Project. "rust-hypervisor-firmware". https://github.com/cloud-hypervisor/rust-hypervisor-firmware.

4 Avi Kivity et al. "kvm: the Linux Virtual Machine Monitor". In: *Proceedings of the Linux Symposium* (2007), pp. 225–230.

5 systemd project. "systemd.resource-control — Resource control unit settings". freedesktop.org manual pages.

6 H. Krawczyk, M. Bellare and R. Canetti. "HMAC: Keyed-Hashing for Message Authentication". RFC 2104 (1997).

7 M. Jones and D. Hardt. "The OAuth 2.0 Authorization Framework: Bearer Token Usage". RFC 6750 (2012).

8 R. Fielding, M. Nottingham and J. Reschke. "HTTP Semantics". RFC 9110 (2022).

9 The Kubernetes Authors. "Resource Bin Packing" (kube-scheduler scoring strategies). kubernetes.io documentation.

10 qemus. "qemu: QEMU in a Docker container". https://github.com/qemus/qemu.

Thank You for Your Attention!